

1. PRODUCT OVERVIEW

Product Name: DevByte

Type: AI-Driven Software Development Lifecycle (SDLC) Automation Platform

Stage: Early product, active internal use and first paying client

Parent Company: ThinkByte Technologies Private Limited

Built By: ThinkByte.AI engineering team

Website: No public website yet (product in early stage)

Source Code: Available on ThinkByte's internal GitHub (repository: Devbyte)

Current Platform: Tested on macOS, Windows support with minor changes (Electron-based)

What DevByte Does

DevByte is an AI-driven SDLC automation platform that streamlines the entire software development lifecycle, from planning through coding to deployment. It uses AI agents to automate code execution, testing, and delivery while keeping humans in control of strategy, planning, and quality review.

Think of it as: "Jira meets AI coding agents." It combines project management (epics, stories, tasks) with autonomous AI-driven code execution, all in one platform.

Core Value Proposition

DevByte automates the execution layer of software development. Instead of developers writing every line of code manually, DevByte's AI agents execute stories and tasks based on well-defined specifications and context. Developers shift their role from writing code to defining what needs to be built, providing context, reviewing output, and approving merges.

Key promise: "A lot of execution or writing the code is going to be taken over by agents. People are going to spend more time in enabling the agentic systems with context and enabling them with the right direction. Once these two are enabled, execution is going to happen automatically."

2. PROBLEM AND MARKET CONTEXT

The Evolution of SDLC

Phase 1 - Waterfall (10+ years ago):

- Entire version (v2) developed, tested, and deployed at once
- Timeline: 3 months to 1 year from version to version
- Problem: Small bug fixes or feature requests could take 6-9 months to ship
- No continuous development or deployment
- Migration and deployment were massive multi-day exercises

Phase 2 - Agile (Last 10 years):

- V2 broken into smaller epics and stories
- Sprint cycles: 1 week, 2 weeks, or monthly
- Deployment at the end of every sprint

- Instead of 10 features after 6 months, 2 features come out every month
- All enterprise companies and startups adopted agile
- Continuous development and deployment became standard

Phase 3 - AI-Driven Development (Now, what DevByte enables):

- AI agents execute coding tasks based on specifications
- Epic that used to take 2 weeks can be done in 2 days
- Support issues, small features, and bug fixes can be automated
- Smaller, simpler tasks achieving 90% one-shot success rate
- Developers spend more time on planning and context, less on writing code
- Parallelization of multiple stories and projects simultaneously

The Key Insight

"The primary aspect of where an agent does well in terms of picking up a task and executing it is purely dependent on whether it has clear understanding of what needs to be developed at a very minute level. If it has clarity on what changes need to be done, it is much more efficient."

This drives two core concepts:

1. Context-Driven Development: Giving the right context (not more, not less) to the agent
2. Test-Driven Development: Agents validate their code through test cases. If test cases pass with 100% coverage, the functionality is working.

3. TWO STRATEGIC DIRECTIONS

Direction 1: Internal Tool

DevByte is used internally by ThinkByte to streamline and empower the team to be more efficient and productive in delivering software products to clients. It serves as ThinkByte's own "agent factory" or "mothership" to automate delivery.

Quote: "We are going to use this internally, and a lot of companies are going to have these internal processes or internal agents with defined processes according to how their organization is defined."

Direction 2: B2B Product

DevByte is being positioned as a B2B product deployed to companies in specific vertical spaces. The initial target vertical is embedded systems engineering.

First paying client: GBS Tech, a company in the railway embedded space working on STM32 controllers.

Quote: "We have actually now a closed client. I'm meeting them in Hyderabad. That is where the first time we are going to launch DevByte in the embedded space."

4. ARCHITECTURE AND TECH STACK

Overall System Architecture

DevByte consists of 4 services and 2 databases:

Service 1: Supabase (Database hosting and backend-as-a-service)

Service 2: FastAPI (Backend application and API layer)

Service 3: Next.js Web Application (Frontend for PMs, analysts, designers)

Service 4: Desktop Application (Electron-based app for developers and coding agents)

Database 1: PostgreSQL (hosted by Supabase, primary relational database)

Database 2: Vector Databases (Weaviate V8 and Neo4j, may consolidate to single DB in future)

Agent Framework: LangGraph (built into the FastAPI layer)

Why Two Frontends (Web App + Desktop App)

The Web App cannot access the developer's local file system, IDE, or project files. It is designed for collaboration on planning, documentation, and project management by analysts, product managers, designers, and scrum masters.

The Desktop App has full access to the local file system. It can navigate project directories, read codebases, execute coding agents, run commands, and push to GitHub. It is designed for developers and AI agents that need to write, test, and commit code.

Web App personas: Analysts, Product Managers, Designers, Scrum Masters

Desktop App personas: Developers, Coding Agents

Desktop App Tech Details

- Built on Electron (cross-platform browser-based desktop app)
- Tested on macOS, Windows support with minor changes
- Integrates with Claude Code SDK (uses Claude API key from OS keychain or API key input)
- Supports both CLI and API key authentication for Claude
- GitHub integration for auto-pushing branches
- Configurable memory allocation and concurrent execution limits
- Security settings: allowed commands, blocked commands, dangerous flags per project
- Built-in terminal for manual Claude CLI invocation
- Supports parallel terminal sessions

5. THREE-TIER KNOWLEDGE SYSTEM

Overview

DevByte organizes knowledge into 3 hierarchical tiers that provide context to AI agents during planning and execution. The quality of this context directly determines the quality of AI output.

Quote: "We feed bad data, the output will by default be bad. We feed bad context, the output will definitely be bad. A lot of data science problems start with data cleaning first and data organization."

Tier 1: Platform Level Knowledge

Definition: Generic knowledge that is common for ANY organization using DevByte. This knowledge does not change from organization to organization.

Examples:

- LangGraph documentation and examples
- Supabase documentation
- Python documentation

- Framework-specific docs and best practices
- Third-party library documentation

This is knowledge provided by DevByte (ThinkByte) as part of the platform itself.

Tier 2: Organization Level Knowledge

Definition: Knowledge specific to a particular organization that applies across all their projects. It defines the organization's standards, preferences, and common practices.

Examples:

- Tech stack preferences (e.g., ThinkByte uses Next.js for frontend, Supabase for database, FastAPI for backend, Weaviate for vector DB)
- Coding standards and conventions
- Database approach (e.g., declarative schema, auto-run migrations, use Supabase CLI, no manual migration files)
- Testing strategy and requirements
- PR and commit guidelines (branch naming, comment conventions)
- Deployment processes
- Common architectural patterns

Key point: Organization-level knowledge helps define initial project-level knowledge for new projects, enabling faster project setup. For example, ThinkByte has a Supabase + Next.js template that new projects can start from.

Tier 3: Project Level Knowledge

Definition: Knowledge specific to a single project that may differ from organization defaults.

Examples:

- Project-specific PRDs (Product Requirements Documents)
- Architecture documents and ADRs (Architecture Decision Records)
- Project briefs
- Scoring logics, matching algorithms, business rules
- UI spec documents (color palette, padding, design standards)
- Technology overrides (e.g., using MongoDB instead of the org-default Supabase)
- Data extraction methods specific to that project

Key point: Project-level knowledge can override organization-level defaults. If an organization defaults to Supabase but a specific project needs MongoDB, the project knowledge overrides the organization default.

Knowledge flows upward: Learnings from individual projects that are found to be common and reusable should be promoted to organization-level knowledge.

6. THREE DEVELOPMENT PHASES

Phase 1: Planning

Starts with: Project understanding

Ends with: Defined epics, stories, and acceptance criteria

What happens:

- Analyst, PM, or Scrum Master builds initial documentation
- AI planning agents assist in creating PRDs, architecture documents, project briefs

- Documentation goes into the knowledge base as project-level context
- Higher-level technical overview of how features will be built
- Defines what needs to be built (not how to code it)

Agent roles in planning: Analyst Agent, PM Agent, Architect Agent, Scrum Master Agent

These agents build docs which then sit in the Documents section of the project and serve as context for coding agents.

Planning happens on the Web App. Execution cannot happen on the Web App.

Phase 2: Execution

Starts with: Approved plan and defined stories

Ends with: Completed code committed to feature branches

What happens:

- Stories from planning are broken into smaller tasks and subtasks
- Each story gets its own Git worktree (isolated copy of the project)
- Coding agents execute tasks within the worktree
- Multiple stories can run in parallel across different worktrees
- For each story, the agent:
 1. Fetches the story from the cloud (Supabase)
 2. Searches knowledge base for relevant ADRs, code examples, and context
 3. Assesses context quality score
 4. Creates a detailed spec document (implementation plan, scope, files to modify, patterns to follow, acceptance criteria)
 5. Human reviews and approves the spec
 6. Agent executes tasks and subtasks sequentially
 7. Each subtask completion is tracked in real-time via MCP integration
 8. Progress is committed at task-by-task level to the story branch
 9. Temporal and episodic memory stores step-by-step learnings for future reuse

Execution happens on the Desktop App.

Phase 3: Testing, Review, and Deployment

Starts with: Completed story in a worktree

Ends with: Code merged to main branch and deployed

What happens:

- End-to-end testing of completed story
- Code quality review
- Git diff comparison and merge conflict detection
- If merge conflicts exist, a specialized merge conflict resolution agent handles them
- Human reviews the changes (new files, modified files, deleted files)
- Once approved, code is pushed to remote repository (GitHub)
- PR is raised
- After PR approval, merge to main/feature/epic branch
- Merging always happens sequentially (even if development was parallel)
- Deployment

7. PARALLELIZATION WITH GIT WORKTREES

How Parallel Development Works

DevByte uses Git Worktrees to enable parallel story development without conflicts.

What is a Git Worktree: A worktree is essentially a duplicate of the entire project in a separate folder. Each story gets its own worktree so agents can code independently without interfering with each other.

Process:

1. Story is picked up from backlog
2. A new worktree is created with all project files duplicated
3. A new branch is created (branch name = story ID)
4. The coding agent develops within this isolated worktree
5. When story is complete, it is committed to its branch
6. Before merging to main, the agent checks for changes made by other stories that were merged after the worktree was created
7. If merge conflicts exist, a merge conflict resolution agent handles them (with human review)
8. After human approval, the story branch is merged to main

Key rules:

- Story development runs in parallel
- Merging always happens sequentially
- Dependent stories have built-in tools to prevent starting before dependencies are complete
- The developer can manage multiple projects, multiple epics, and multiple stories all in parallel from a single desktop app interface

8. AI AGENT ARCHITECTURE

Planning Agents

The planning phase uses multiple specialized agents:

- Analyst Agent: Gathers and organizes requirements
- PM Agent: Defines product specifications and priorities
- Architect Agent: Creates technical architecture and design documents
- Scrum Master Agent: Breaks work into epics and stories, manages workflow

These agents create documentation that serves as context for coding agents.

Coding Agent Workflow

The coding agent follows a multi-step workflow for each story:

- Step 1: Discovery - Fetch story details, gather requirements, gather context
- Step 2: Knowledge Search - Look up ADRs, code examples, patterns from all 3 knowledge tiers
- Step 3: Context Quality Assessment - Score how much quality context is available
- Step 4: Spec Creation - Create detailed implementation plan (rationale, scope, files, patterns, testing, acceptance criteria)
- Step 5: Human Review and Approval of Spec
- Step 6: Task Execution Loop - Execute each task and subtask, update status in real-time via MCP
- Step 7: Validation - Validate completed work against acceptance criteria
- Step 8: Memory Storage - Upload learnings to Neo4j for future reuse
- Step 9: Commit and Push - Commit to story branch, push to remote

Memory Management

DevByte implements temporal and episodic memory using Neo4j graph database:

- Every change, decision, and learning is stored step by step
- When revisiting a story or feature, the agent has previous context and learnings
- Pattern discovery identifies common code patterns and stores them as reusable knowledge
- Vector embeddings enable semantic search across the knowledge base

Quote: "Right now, there is no memory management in Claude Code. It's all docs that it fetches, checks, and writes. So DevByte should definitely be on par with what we have and should only get better."

Merge Conflict Resolution Agent

A specialized agent that:

- Has context of what was done in both conflicting stories
- Understands the intent of changes in each story
- Resolves merge conflicts automatically
- Presents resolution for human review before final merge

MCP (Model Context Protocol) Integration

DevByte has MCPs connected to coding agents that:

- Automatically update task and subtask status in the backend in real-time
- Track exactly how much time each task/subtask takes
- Show live progress of where the agent is in the workflow
- Enable the web app to display real-time execution status

9. USER INTERFACE AND WORKFLOW

Web App Features

- Dashboard showing all projects
- Knowledge base management (organization and project level)
- Document creation and editing (PRDs, architecture docs, project briefs)
- Epic and story management
- Integration with Jira, Linear, and other project management tools
- Planning collaboration across team members
- AI planning agents accessible from the web

Desktop App Features

- Multi-project management from a single interface
- Project linking (connect cloud project to local file path)
- Settings configuration:
 - Claude Code integration (CLI or API key)
 - GitHub integration for auto-push
- Memory allocation limits
- Concurrent execution limits
- Security settings (allowed/blocked commands, dangerous flags)
- Story execution with real-time logs

- Two modes: Automation mode (logs only) and Interactive mode (Claude CLI-like interaction, coming soon)
- Git worktree management
- Code review interface (view new, modified, deleted files before approval)
- Built-in terminal for manual Claude CLI invocation
- Parallel terminal sessions
- Workflow customization per project
- Real-time task and subtask tracking

Workflow Customization

DevByte supports customizable workflows:

- Different workflows can be defined for different projects
- Organizations can define their own workflow that best fits their process
- Future: Visual workflow builder for drag-and-drop workflow creation
- Current: One default workflow defined (plan -> code -> review -> merge)

10. INTEGRATIONS

Current Integrations

- Claude Code SDK (primary AI coding engine)
- Supabase (database and backend services)
- GitHub (repository management, branch creation, push, PR)
- Neo4j (graph database for memory and knowledge management)
- Weaviate V8 (vector database for semantic search)
- MCP Servers (real-time status updates)

Planned Integrations

- Jira (import existing epics, stories, and project knowledge)
- Linear (project management sync)
- Other LLM models beyond Claude (similar to Cursor's multi-model approach)
- Support for multiple AI model tiers (start with Opus, downgrade to cheaper models as usage increases)

LLM Strategy

Quote: "LLM is an open space. We are using Claude SDK. The Claude SDK needs a Claude API key, but we can also use other models with it. Think just like Cursor - you start with Opus, then you move to lower grade models as you start using more."

For B2B clients: DevByte handles LLM costs. Clients don't need to purchase AI models separately. They just set up GitHub and start using the platform.

11. ONE-SHOT EXECUTION

Current State of AI-Driven Coding

Based on team observations shared during the demo:

- Simpler tasks: ~90% one-shot success rate (agent completes the task correctly without human intervention)

- Complex tasks: Lower one-shot rate, requiring manual intervention when context is large or integrations with existing codebase are complex
- Tasks that work well one-shot: Bug fixes, small feature upgrades, simple scripts, isolated components
- Tasks that struggle: New features requiring deep integration with existing codebase, large context requirements, complex multi-system changes

Quote: "At the end of the day, the goal for us or any company is to be able to drive execution through agents. How much percentage can be done by one-shot? This is gonna keep improving."

12. COMPETITIVE LANDSCAPE

Known Competitors

Competitor 1: (Y Combinator-funded US company, name discussed but not clearly stated in transcript)

- Similar concept to DevByte
- Recently discovered (day before demo)
- Already funded by Y Combinator
- Very early stages
- Uses Claude and CodeSandbox

Other competitive tools mentioned:

- Cursor (AI-powered IDE)
- Claude Code CLI (coding assistant)
- Jira (project management, not AI-driven development)

DevByte's Competitive Strategy

1. Niche Focus: Target the embedded systems vertical specifically (starting with railway embedded, STM32 controllers)
2. Internal Dogfooding: Use DevByte for ThinkByte's own projects to continuously improve the product
3. Bootstrap: No external funding sought. Self-funded through revenue from client projects.
4. Context Engineering: Focus on superior context management (3-tier knowledge system) as the key differentiator
5. Memory Management: Temporal and episodic memory that existing tools like Claude Code don't have
6. Parallelization: Multi-project, multi-story parallel execution capability

Quote: "The way for us to win or take it forward is to focus on a specific niche."

13. CLIENTS AND TRACTION

First Paying Client

Client: GBS Tech

Industry: Railway embedded systems

Technology: STM32 controllers

Status: Closed deal, paid customer

Next Step: In-person meeting in Hyderabad (Thursday after the demo call)

Significance: First external deployment of DevByte in the embedded space

Internal Usage

DevByte is being used internally by ThinkByte for their own software delivery across client projects. The goal is to make

internal development more efficient and faster while simultaneously improving the product through real-world usage.

14. TEAM AND LEADERSHIP

Demo Presenter

Vinay Kovvuri (also known as Vinay Kumar Reddy Kovvuri)

Role: Co-Founder, ThinkByte Technologies / DevByte product lead

Education: BITS Pilani

Experience: 10+ years in AI/ML product leadership

Also leads: ThinkByte.AI, GrowthByte, SeoByte

Engineering Team Members (Present in Demo)

- Bodhisatwa Chakraborty (Developer)
- Utkarsh Jain (Developer)
- Satish Chitturi (Developer)
- Anupam Gevariya (Developer)
- Samarpith Rout (Developer)
- Shivani Singamsetty (Developer)
- Sushruth Suresh (Developer)
- Shekhar (Developer, joined late)
- Bhargav Yerraguntla Thirumala Venkata (Developer)

Other Key People

SriHarsha Thota: Co-Founder, ThinkByte Technologies (not present in this demo but co-leads the company)

Raghu Gorrela: Co-Founder of GrowthByte, handles global client engagements (BITS Pilani + XLRI Jamshedpur)

15. RELATIONSHIP TO THINKBYTE GROUP

The ThinkByte Product Family

DevByte is the fourth product/brand under the ThinkByte Technologies umbrella:

Brand 1: ThinkByte.AI (thinkbyte.ai)

Focus: AI consulting, LangGraph development, multi-agent AI systems

Target: Companies building AI products

Brand 2: GrowthByte (growthbyte.ai)

Focus: AI-powered marketing services

Target: Brands needing marketing growth

Brand 3: SeoByte (seobyte.ai)

Focus: AI-powered SEO automation platform

Target: SEO agencies, businesses scaling organic traffic

Brand 4: DevByte (no public website yet)

Focus: AI-driven SDLC automation platform

Target: Internal use + B2B (embedded systems vertical initially)

All four brands are operated under ThinkByte Technologies Private Limited (CIN: U62013KA2023PTC179630), headquartered at No. 235, 13th Cross, Hoyasala Nagar 2nd Stage, Bangalore, Karnataka 560038.

16. FUTURE ROADMAP

Immediate Next Steps

1. Make current DevByte more seamless and functioning
2. Start internal daily usage across ThinkByte projects
3. Load existing project documentation (PRDs, architecture docs) into DevByte
4. Launch DevByte with GBS Tech (first external client, embedded space)
5. Demo session with loaded project context and live story execution

Planned Features

- Interactive mode for desktop app (Claude CLI-like interaction, not just automation logs)
- Visual workflow builder for custom SDLC workflows per project/organization
- Jira and Linear integrations
- Epic and story creation agents (currently manual, will be AI-assisted)
- Multi-model support (beyond Claude, similar to Cursor)
- Brownfield project support with BMAT flows for generating documentation from existing codebases
- Windows support refinement
- Token efficiency optimization
- UI spec document support for frontend component development
- Workflow customization per project and organization

Open Challenges

- Token efficiency and cost management for AI model usage
- Brownfield projects where documentation doesn't exist (using BMAT flows to create docs)
- Complex UI development requiring many adjustments (solution: componentize into smaller stories with UI spec docs)
- Accuracy of AI output for complex tasks (comparable to Claude Code currently, expected to improve with better models)
- Windows compatibility (Electron-based, should work with minor changes)

17. KEY CONCEPTS AND TERMINOLOGY

SDLC: Software Development Life Cycle - the entire process from planning to deployment

Epic: A large body of work that can be broken into smaller stories

Story: A specific piece of work that delivers a single feature or change

Sprint: A fixed time period (1-4 weeks) in which a set of stories are completed

One-Shot: When an AI agent completes a task correctly on the first attempt without human intervention

Git Worktree: An isolated copy of a project in a separate folder, enabling parallel development

Context-Driven Development: Providing precise, accurate context to AI agents for better output

Test-Driven Development: Writing test cases first, then having agents code until tests pass

MCP: Model Context Protocol - enables real-time communication between AI agents and backend systems

PRD: Product Requirements Document

ADR: Architecture Decision Record

Brownfield: An existing project/codebase (as opposed to greenfield/new projects)

Spec Document: Detailed implementation plan created by planning agents before coding begins

18. SOURCES

- Internal DevByte demo call recording and transcript (presented by Vinay Kovvuri to ThinkByte engineering team)
- ThinkByte Technologies corporate registration (CIN: U62013KA2023PTC179630)
- ThinkByte.AI company documentation and case studies
- DevByte GitHub repository (internal, ThinkByte organization)

Document compiled: February 10, 2026

Based on internal demo call transcript. DevByte does not have a public website at this time.